

OLAP Database Optimization Strategy

The main challenge with Stripe's OLAP database is processing millions of transactions. This document describes the strategies implemented to optimize analysis performance on such a large volume of data.

1. Query and Join Optimization

The first step is to reduce the total volume of data scanned to access it. To achieve this, the tables were designed to facilitate large-scale joins. Indexes were therefore created on the foreign keys of `fact_transaction` (using `customer_key`, `merchant_key`, `geography_key`, `time_key`, and `payment_key`).

It is also important to be able to easily select the time range required for analysis. This is why the `dim_time` dimension breaks down time so precisely, for example, into days, months, and years. Thus, the `fact_transaction` table is partitioned by year and month using the `time_key` column. This allows the database to scan only the partition corresponding to the desired period.

Finally, to optimize computing power when performing a selection within a selection, it's best to avoid using subqueries. These will execute for each row. Instead, use joins or Common Table Expressions, which offer greater speed.

2. Materialized Views

Materialized views have also been created. These are queries whose results are stored to facilitate analysis. They allow for quick answers to key business questions. They are updated nightly via queries recorded in the `'queries_olap.sql'` file.

The first materialized view is `summary_daily_revenue`, which answers questions about Stripe's performance on a given day. Each column corresponds to the answer to a question:

- What is the total revenue for a day?
- How many transactions occurred in a day?
- What is the average transaction amount? How many failed attempts?
- How many frauds were detected that day?

The second visualization is `'summary_fraud_by_country'`, which focuses on fraud by country. The columns answer the questions:

- How many frauds occurred in total in this country?
- What is the total amount involved?
- What is the fraud rate per country?

The final visualization is `'summary_merchant_performance'`, which focuses on merchants. The columns answer the following questions:

- What is the total revenue of a particular merchant?
- What is the total number of their transactions?
- What is their refund rate?

- What is their fraud rate?

Thus, query response time is reduced through indexing and time partitioning. Replacing subqueries with joins optimizes computing power, and materialized views enable rapid analysis of Stripe's millions of database rows.